

Exercícios de Python

Bloco 1 — Sintaxe básica e tipagem dinâmica

1. Peça dois números e imprima soma, subtração, multiplicação, divisão real (/) e divisão inteira (//), além do resto (%).
2. Troque o valor de duas variáveis sem usar variável auxiliar (dica: `a, b = b, a`).
3. Dado um número, verifique se é par ou ímpar usando operador ternário ("par" if `x % 2 == 0` else "ímpar").
4. Formate uma string usando f-string: dado nome e idade, imprima "Fulano tem 30 anos".
5. Faça um programa que recebe uma string e imprime ela invertida (`s[::-1]`) — pratique slicing.

Bloco 2 — Estruturas de dados nativas

1. Crie uma lista de 10 números e imprima só os pares usando list comprehension.
2. Dado um dicionário de nomes e notas, imprima quem tirou acima da média usando `.items()`.
3. Use uma tupla para retornar múltiplos valores de uma função (ex: mínimo e máximo de uma lista).
4. Crie um set a partir de uma lista com duplicados e mostre a diferença de tamanho entre eles.
5. Pratique `zip()`: dadas duas listas (nomes e notas), imprima os pares combinados.
6. Pratique `enumerate()`: percorra uma lista imprimindo índice e valor.
7. Unpacking avançado: `a, *meio, b = [1, 2, 3, 4, 5]` — imprima `a`, `meio` e `b`.

Bloco 3 — Funções

1. Escreva uma função com parâmetro default: `def saudacao(nome, msg="Olá")`.
2. Escreva uma função com `*args` e `**kwargs` que soma números e imprime dados extras.
3. Escreva uma função que retorna outra função (closure) — ex: um multiplicador configurável.
4. Use `lambda` junto com `map()`/`filter()` para transformar uma lista de números.
5. Pratique type hints: `def soma(a: int, b: int) -> int:`

Bloco 4 — Controle de fluxo e exceções

1. Reescreva um switch/case de C usando `match/case` (novidade do Python 3.10+).
2. Faça um programa com `try/except/else/finally` tratando `ZeroDivisionError` e `ValueError`.

3. Crie uma exceção customizada (class MinhaExcecao(Exception)) e dispare com raise.

Bloco 5 — POO em Python (comparando com Java)

1. Crie uma classe Pessoa com `__init__`, um método e o self explícito (diferente do this implícito de Java).
2. Implemente `__str__` e `__repr__` na classe (equivalente ao toString() do Java).
3. Pratique herança: class Aluno(Pessoa) sobrescrevendo um método com super().__init__().
4. Use @property para criar um getter/setter pythônico (sem os métodos getX()/setX() de Java).
5. Crie uma classe com um método @staticmethod e outro @classmethod — explique a diferença entre eles.
6. Implemente uma interface "informal" usando duck typing ou ABC/abstractmethod.
7. Sobrecarregue operadores: implemente `__add__` numa classe Vetor2D.

Bloco 6 — Módulos e boas práticas

1. Separe seu código em dois arquivos e use import entre eles.
2. Use if `__name__ == "__main__"`: e explique por que isso existe (não tem equivalente direto em Java).
3. Use o módulo random para simular um jogo de dados, e datetime para calcular idade a partir de uma data de nascimento.